LATENTLAB

Recurrent Latent-Space Reasoning: Can an AI Think Longer Without Talking Longer?

Central claim

Repeated hidden-state computation can increase inference effort without requiring a longer verbal reasoning trace. LATENTLAB demonstrates this mechanism-level possibility with a small deterministic recurrent substrate. The claim is deliberately two-sided: additional recurrent updates can change an answer, but they are not guaranteed to remain useful indefinitely.

Mechanism

The input is a binary grid with a start, goal, open cells, and walls. The system initializes a fixed-size integer state with distance zero at the start and unknown values elsewhere. One synchronous update examines the previous state and propagates the smallest known neighbor distance by one grid edge. Repeating the same local rule `N` times lets task-relevant information travel farther without emitting intermediate words:

```
input grid -> fixed-size state -> N synchronous local
updates -> goal estimate
```

The visible state is not a guessed neural interpretation: it is the actual task-specific integer state used by this algorithm. Reasoning depth is therefore an exact count of executed updates.

Interactive evidence

In **Depth Matters**, the goal is six edges from the start. At depth 4, information has not propagated far enough, so the recurrence reports "not yet reached" while the completed reference says the goal is reachable. At depth 6, the live state reaches the goal with distance 6 and agrees with the reference. This is a bounded causal example: changing one control changes the number of executed updates and the observed state.

In **More work, same answer**, a wall disconnects the goal. The reachable component reaches a fixed point at step 3. Requesting updates through depth 16 increases the operation proxy, but every later state and output remains unchanged. That is saturation - not a claim that the system becomes less intelligent.

Independent reference

A separate breadth-first search (BFS) computes reachability and shortest distance. The recurrent engine never imports or calls BFS; the experiment runner invokes them as sibling computations and displays the comparison. BFS evaluates the estimate but does not feed or guide recurrent inference.

Published research connection

Several research directions spend inference computation without requiring a natural-language token for every intermediate update. Geiping et al. study test-time depth through a repeated recurrent block [S1]. Coconut feeds a model's last hidden state back as a continuous input representation [S2]. These are distinct mechanisms, not interchangeable names.

BDH provides architectural context: its authors describe learned graph/local dynamics and an evolving associative state during inference [S3]. BDH-CQ's authors describe demonstrations updating recurrent contextual memory, followed by iterative computation in a separate latent query workspace before decoding [S4]. The shared idea with LATENTLAB is only repeated internal computation before output.

LATENTLAB is **not** BDH or BDH-CQ. It has no learned parameters, neural activations, in-context learning, language model, candidate ranking, or ARC evaluation. Exact BDH-CQ update rules and dimensions remain proprietary, and published BDH/BDH-CQ results were not reproduced by this project.

Evidence maturity and limits

OUR EXPERIMENT: live recurrence, BFS reference, deterministic traces, 10-fixture/80-case reproducibility corpus, tests, and checksums. **AUTHOR-REPORTED PREPRINT:** BDH, BDH-CQ, recurrent-depth, and Coconut mechanisms/results. **CONCEPTUAL COMPARISON:** the original simplified diagram and cross-system explanation.

The substrate is algorithmic rather than learned; tasks are tiny symbolic grids; there is no language reasoning, calibrated confidence, participant study, or evidence about frontier-model cognition. The operation proxy - requested depth times traversable cells - is not FLOPs, latency, energy, token count, or price. The small hand-constructed corpus is not a benchmark.

Takeaway

More internal recurrent computation can change an answer without producing more reasoning text - but additional computation can also saturate.

Primary sources: S1 Geiping et al., 2025; S2 Hao et al., 2024; S3 Kosowski et al., 2025; S4 Engdahl et al., 2026.